SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems**

COURSE CODE: **CSA0509**

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Short Answer Text (High)

Assessment Tool Weightage: 40%

Q. No	Question	Bloom's Level
1	Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.	BL1 – Remember
2	Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.	BL2 – Understand
3	Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.	BL2 – Understand
4	Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.	BL2 – Understand
5	Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.	BL1 – Remember

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	30%	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Linkages	25%	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Organization	20%	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Integration of Literature	15%	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity	10%	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ASSESMENT TOOL – 01

Q1. Explain Schema and Instance with examples. Distinguish between Logical Schema, Physical Schema, and View Schema.

Introduction

A Database Management System (DBMS) organizes and manages data efficiently. To understand how a database works, it is important to know the concepts of Schema, Instance, and the Three-Schema Architecture (Logical, Physical, and View Schema). These concepts help in designing, storing, and accessing data efficiently.

1. Schema

Definition

A Schema is the logical blueprint or overall structure of a database. It defines how the database is organized by specifying the tables, attributes (columns), data types, constraints, primary keys, foreign keys, and relationships among tables. Since it represents the database design, it changes very rarely.

Characteristics

- Defines the structure of the database.
- Created during database design.
- Remains mostly unchanged.
- Ensures consistency in data organization.

Example

Student Table Schema

Attribute	Data Type	Description
Student_ID	INT	Unique ID of the student (Primary Key)
Name	VARCHAR(50)	Student Name
Age	INT	Student Age
Department	VARCHAR(30)	Department Name

The above table structure is called the Schema because it defines how student information will be stored.

2. Instance

Definition

An Instance is the actual collection of data stored in the database at a particular moment. It changes whenever records are inserted, updated, or deleted.

Characteristics

- Contains real-time data.
- Changes frequently.
- Represents the current state of the database.

Example

Student Table Instance

Student_ID	Name	Age	Department
101	Kadhir	18	CSE
102	Rahul	19	ECE
103	Anitha	18	AI & DS

The above records represent the Instance of the Student table.

3. Difference Between Schema and Instance

Feature	Schema	Instance
Meaning	Structure or blueprint of a database	Actual data stored in the database
Nature	Static (changes rarely)	Dynamic (changes frequently)
Created During	Database design	Database operation
Contains	Tables, attributes, relationships, constraints	Records (rows)
Example	Student (Student_ID, Name, Age, Department)	(101, Kadhir, 18, CSE)

4. Types of Schema

A. Logical Schema

Definition

The Logical Schema describes what data is stored in the database and how different tables are related. It does not describe the physical storage of data.

Includes

- Tables
- Attributes
- Primary Keys
- Foreign Keys
- Relationships
- Constraints

Example

A College Database contains:

- Student (Student_ID, Name, Age)
- Course (Course_ID, Course_Name)
- Enrollment (Student_ID, Course_ID)

These tables and their relationships together form the Logical Schema.

Importance

- Represents the overall database design.
- Independent of physical storage.
- Used by database designers.

B. Physical Schema

Definition

The Physical Schema describes how the database is physically stored in memory or on storage devices. It includes storage structures and techniques used to improve database performance.

Includes

- Storage location
- Files
- Indexes
- Data blocks
- Access methods

Example

The Student table is stored in a database file, and an index is created on the Student_ID column to make searching faster.

Importance

- Improves query performance.
- Optimizes storage utilization.
- Managed by the Database Administrator (DBA).

C. View Schema

Definition

A View Schema defines what part of the database is visible to a particular user. Different users can have different views depending on their roles and permissions.

Example:

Teacher View

Student_ID	Name	Marks

Student View

Name	Marks	Attendence

Different users access only the information they are authorized to see.

Importance

- Improves security.
- Protects confidential data.
- Simplifies database usage for users.

5. Difference Between Logical, Physical, and View Schema

Logical Schema	Physical Schema	View Schema
Describes what data is stored.	Describes how data is stored.	Describes what data users can see.
Focuses on database design.	Focuses on storage and performance.	Focuses on user access.
Used by database designers.	Used by DBAs.	Used by end users and applications.
Independent of storage details.	Depends on storage devices and file organization.	Independent of physical storage.

Real-Life Example

Consider a College Management System:

- **Schema:** The design includes Student, Faculty, Course, and Department tables.
- **Instance:** Student Kadhir with Student_ID 101 is stored in the Student table.
- **Logical Schema**: Defines the relationship between Student and Course through Enrollment.
- **Physical Schema:** Stores the tables in database files and creates indexes for faster searching.
- **View Schema:** Students can view only their marks, while faculty can view all students' records.

Conclusion

A Schema is the blueprint that defines the structure of a database, whereas an Instance is the actual data stored at a particular time. The Logical Schema defines the database structure, the Physical Schema explains how data is stored internally, and the View Schema controls what data each user can access. Together, these schema levels ensure efficient database design, secure data access, and improved performance.

Q2. Identify the Entities, Attributes, and Relationships for a Bank Management System and Construct its ER Diagram.

Introduction

A **Bank Management System** is a database application used to manage customer information, bank accounts, transactions, loans, and employees efficiently. An **Entity-Relationship (ER) Diagram** is used to represent the entities, attributes, and relationships involved in the system. It helps in designing a well-structured database.

1. Entities and Their Attributes

Entity 1: CUSTOMER

Attributes

- Customer_ID (Primary Key)
- Name
- Date_of_Birth
- Gender
- Phone_Number
- Email
- Address

Entity 2: ACCOUNT

Attributes

- Account_No (Primary Key)
- Account_Type
- Balance
- Opening_Date
- Customer_ID (Foreign Key)

Entity 3: TRANSACTION

Attributes

- Transaction_ID (Primary Key)
- Transaction_Date
- Transaction_Type
- Amount
- Account_No (Foreign Key)

Entity 4: LOAN

Attributes

- Loan_ID (Primary Key)
- Loan_Type
- Loan_Amount
- Interest_Rate
- Loan_Status
- Customer_ID (Foreign Key)

Entity 5: BRANCH

Attributes

- Branch_ID (Primary Key)
- Branch_Name
- Location
- IFSC_Code

Entity 6: EMPLOYEE

Attributes

- Employee_ID (Primary Key)
- Employee_Name
- Designation
- Salary
- Phone_Number
- Branch_ID (Foreign Key)

2. Relationships

CUSTOMER — ACCOUNT

- One customer can have **many accounts**.
- Each account belongs to **one customer**.
- **Relationship:** Owns
- **Cardinality:** One-to-Many (1:M)

ACCOUNT — TRANSACTION

- One account can have **many transactions**.
- Each transaction belongs to **one account**.
- **Relationship:** Performs
- **Cardinality:** One-to-Many (1:M)

CUSTOMER — LOAN

- One customer can apply for **multiple loans**.
- Each loan belongs to **one customer**.
- **Relationship:** Applies
- **Cardinality:** One-to-Many (1:M)

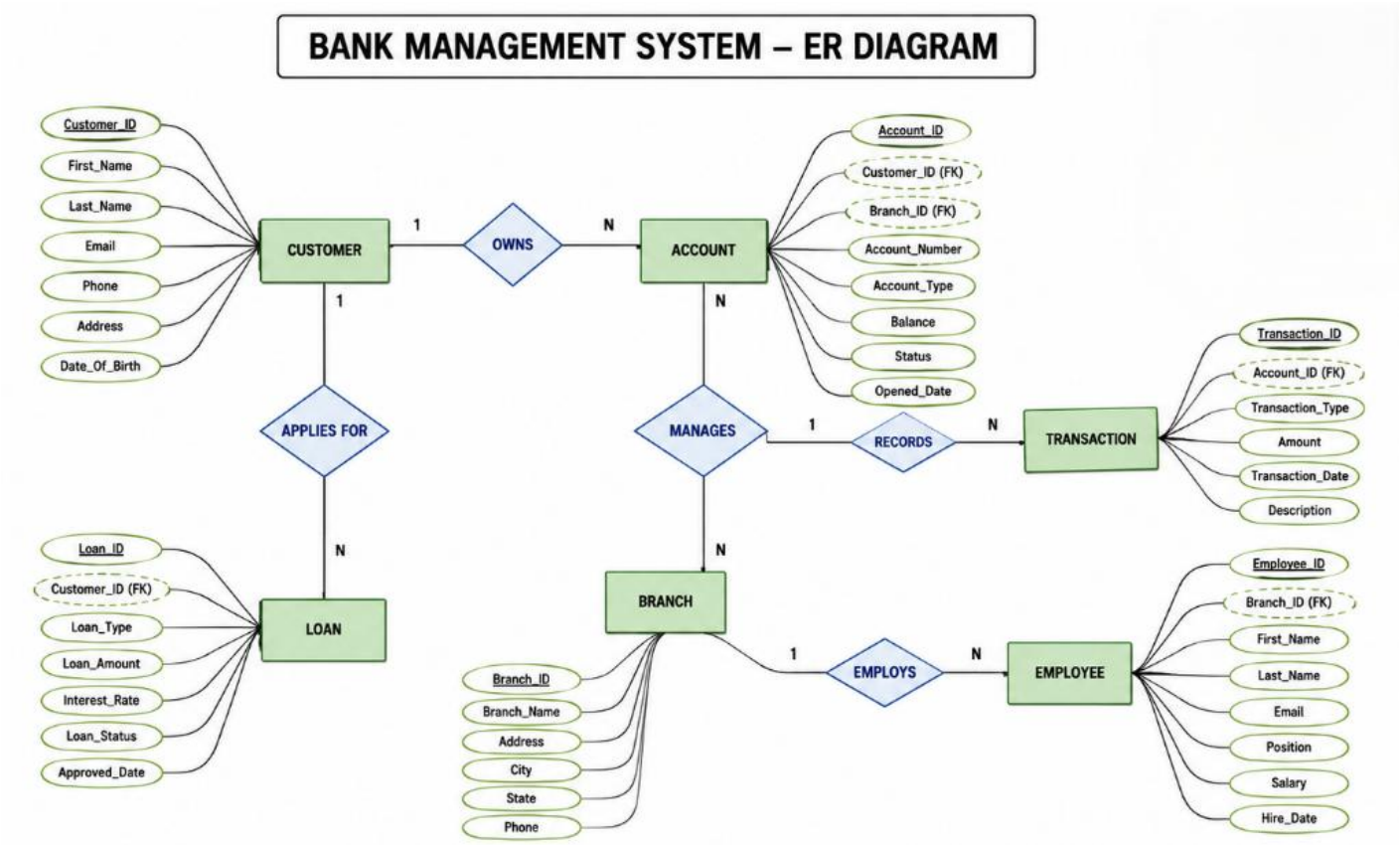
BRANCH — EMPLOYEE

- One branch has **many employees**.
- Each employee works in **one branch**.
- **Relationship:** Employs
- **Cardinality:** One-to-Many (1:M)

BRANCH — ACCOUNT

- One branch manages **many accounts**.
- Each account is maintained by **one branch**.
- **Relationship:** Maintains
- **Cardinality:** One-to-Many (1:M)

3. ER Diagram (Text Representation)



4. Cardinality Summary

Relationship	Cardinality
Customer → Account	1 : M
Account → Transaction	1 : M
Customer → Loan	1 : M
Branch → Employee	1 : M
Branch → Account	1 : M

5. Importance of the ER Diagram

- Identifies all entities involved in the banking system.
- Defines relationships between entities.
- Eliminates data redundancy.
- Improves data integrity and consistency.
- Serves as the foundation for designing relational database tables.
- Makes database development and maintenance easier.

Conclusion

The ER Diagram of a Bank Management System consists of six major entities: **Customer, Account, Transaction, Loan, Branch, and Employee**. Their attributes and relationships accurately represent the banking process. By defining primary keys, foreign keys, and cardinality, the ER model provides a clear and efficient blueprint for implementing a reliable database system.

Q3. Explain the Extended Entity-Relationship (EER) Model. Describe the Concepts of Generalization, Specialization, and Aggregation.

Introduction

The **Extended Entity-Relationship (EER) Model** is an advanced version of the **Entity-Relationship (ER) Model**. It extends the ER model by introducing additional concepts such as **Generalization**, **Specialization**, and **Aggregation**. These concepts help represent complex real-world situations more accurately and make database design more flexible, efficient, and easier to maintain.

1. Extended Entity-Relationship (EER) Model

Definition

The **Extended Entity-Relationship (EER) Model** is an enhanced data model that extends the traditional ER model by supporting advanced concepts like inheritance, superclass-subclass relationships, specialization, generalization, and aggregation.

It is mainly used to model complex databases where entities share common properties or have specialized characteristics.

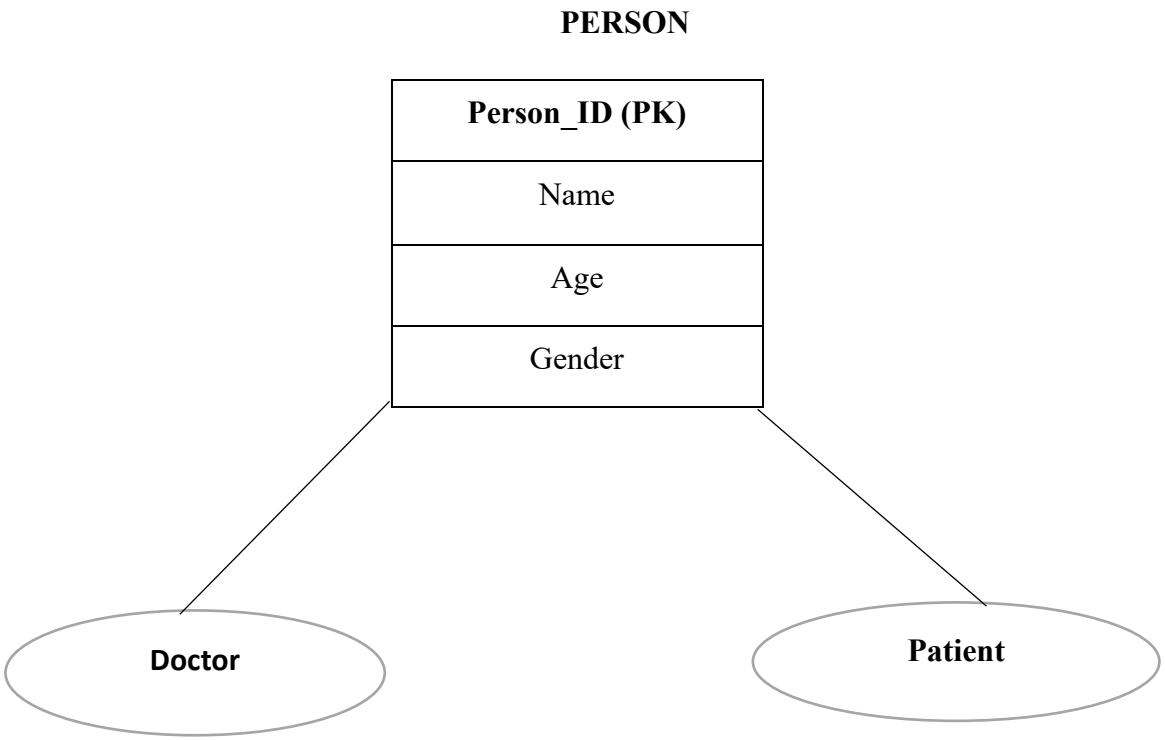
Features of the EER Model

- Extends the traditional ER model.
- Supports inheritance between entities.
- Reduces data redundancy.
- Represents real-world objects more accurately.
- Improves database organization and scalability.
- Makes database design easier to understand and maintain.

Example

In a **Hospital Management System**:

- **Person** is a common entity.
- **Doctor** and **Patient** inherit common attributes from Person.



Here, **Doctor** and **Patient** inherit the attributes of **Person**, demonstrating the EER model.

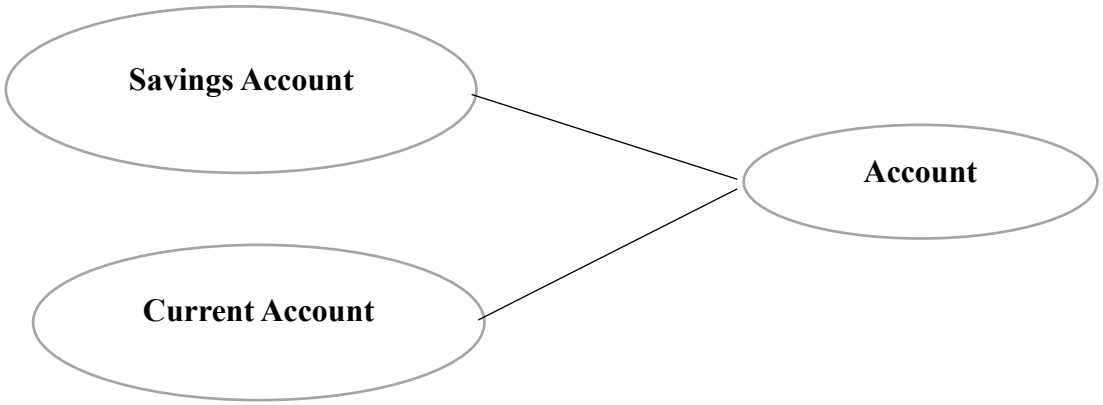
2. Generalization

Definition

Generalization is the process of combining two or more lower-level entities that have common characteristics into a single higher-level entity called a **Superclass**.

It follows a **Bottom-Up Approach**, where similar entities are merged into one generalized entity.

Example



Both **Savings Account** and **Current Account** have common attributes such as:

- Account Number
- Balance
- Opening Date

These common features are combined into the superclass **Account**.

Advantages

- Eliminates duplicate data.
- Reduces redundancy.
- Improves database organization.
- Makes maintenance easier.

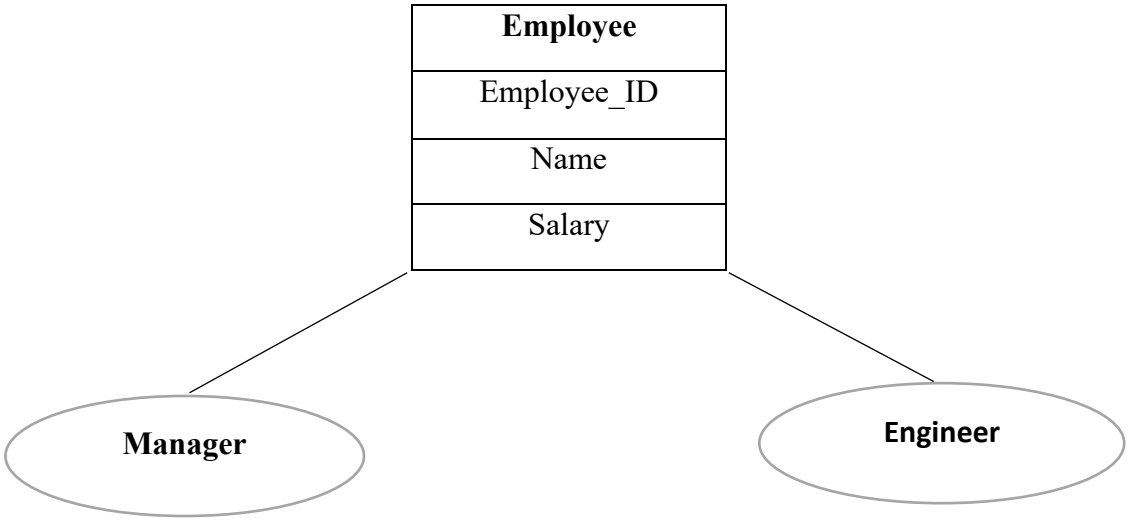
3. Specialization

Definition

Specialization is the process of dividing one higher-level entity into multiple lower-level entities based on their unique characteristics.

It follows a **Top-Down Approach**, where a general entity is divided into more specific entities.

Example



Both **Manager** and **Engineer** inherit common attributes from **Employee**, while each has its own unique properties.

Advantages

- Represents specialized information.
- Improves data organization.
- Avoids storing unnecessary attributes.
- Makes the database more flexible.

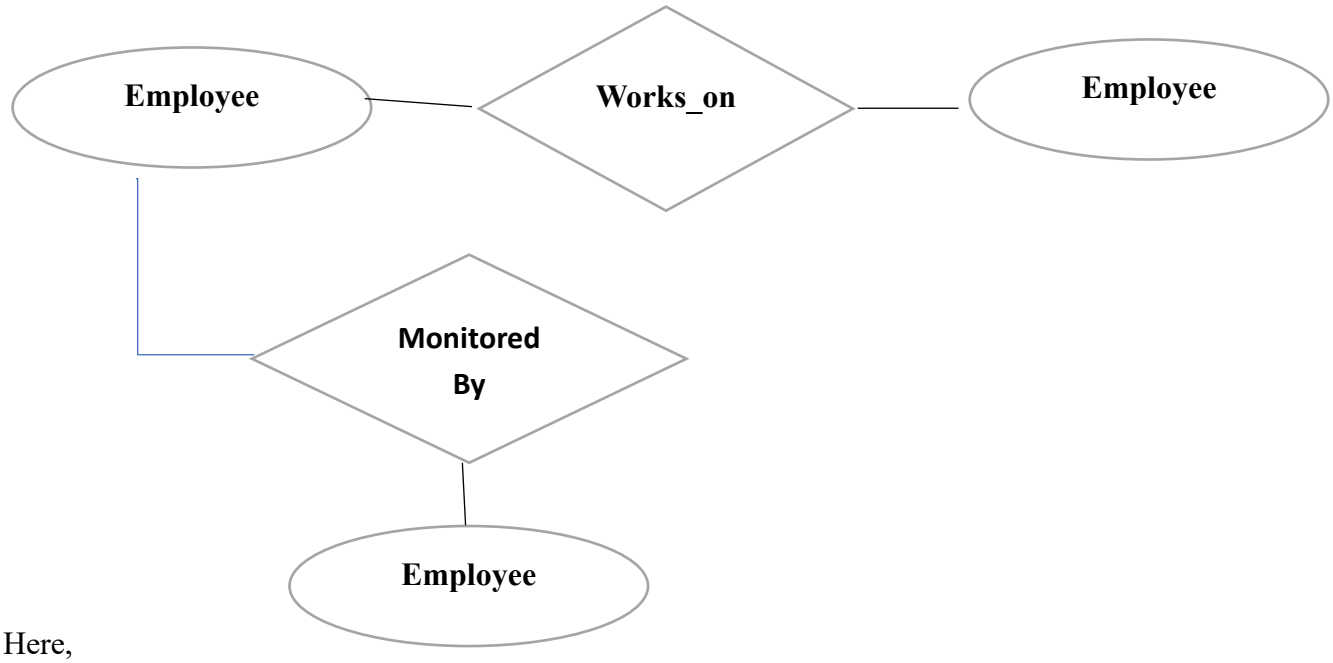
4. Aggregation

Definition

Aggregation is a technique used to represent a **relationship between a relationship and another entity**. It treats a relationship as a higher-level entity so that it can participate in another relationship.

Aggregation is useful when one relationship itself needs to be associated with another entity.

Example



Here,

- Employee works on a Project.
- The relationship **Works_On** is monitored by a Department.

Since the relationship itself participates in another relationship, **Aggregation** is used.

Advantages

- Represents complex relationships.
- Improves database modeling.
- Makes the database more realistic.
- Simplifies complex designs.

5. Difference Between Generalization and Specialization

Generalization	Specialization
Bottom-Up approach	Top-Down approach
Combines lower-level entities into one superclass	Divides one superclass into multiple subclasses
Removes duplication	Adds detailed classification
Example: Savings Account and Current Account → Account	Example: Employee → Manager and Engineer

6. Comparison of EER Concepts

Concept	Purpose	Example
EER Model	Extends the ER model with advanced features	Person → Doctor, Patient
Generalization	Combines similar entities into a superclass	Savings Account + Current Account → Account
Specialization	Divides a superclass into subclasses	Employee → Manager, Engineer
Aggregation	Represents a relationship as an entity	Employee Works_On Project monitored by Department

Applications of the EER Model

- Hospital Management System
- Banking System
- University Management System
- Library Management System
- Online Shopping System
- Human Resource Management System

Conclusion

The **Extended Entity-Relationship (EER) Model** enhances the traditional ER model by introducing **Generalization**, **Specialization**, and **Aggregation**. These concepts help represent complex real-world scenarios efficiently, reduce redundancy, improve database organization, and simplify database design. Therefore, the EER model is widely used for designing modern database systems.

Q4. Design an EER Diagram for a Hospital Management System Incorporating Inheritance and Weak Entities.

Introduction

A **Hospital Management System (HMS)** is a database system used to manage hospital operations such as patient registration, doctor information, appointments, treatments, medical records, and billing. The **Extended Entity-Relationship (EER) Model** is used because it supports advanced concepts such as **inheritance (generalization/specialization)** and **weak entities**, making the database more efficient and realistic.

1. Entities and Attributes

Superclass: PERSON

Attributes

- Person_ID (Primary Key)
- Name
- Age
- Gender
- Phone_Number
- Address

Subclass: DOCTOR (inherits PERSON)

Attributes

- Doctor_ID (Primary Key)
- Specialization
- Qualification
- Experience
- Salary

Subclass: PATIENT (inherits PERSON)

Attributes

- Patient_ID (Primary Key)
- Blood_Group
- Disease
- Admission_Date
- Discharge_Date

Entity: DEPARTMENT

Attributes

- Department_ID (Primary Key)
- Department_Name
- Location

Entity: APPOINTMENT

Attributes

- Appointment_ID (Primary Key)
- Appointment_Date
- Appointment_Time
- Status

Weak Entity: MEDICAL_RECORD

A **Medical_Record** cannot exist without a **Patient**. Therefore, it is a **Weak Entity**.

Attributes

- Record_No (Partial Key)
- Diagnosis
- Treatment
- Prescription
- Test_Result

2. Relationships

PERSON → DOCTOR

- Inheritance (Specialization)
- One Person can become one Doctor.

PERSON → PATIENT

- Inheritance (Specialization)
- One Person can become one Patient.

DOCTOR → DEPARTMENT

- A department has many doctors.
- Each doctor works in one department.
- Cardinality: **1 : M**

DOCTOR → APPOINTMENT

- One doctor attends many appointments.
- Each appointment belongs to one doctor.
- Cardinality: **1 : M**

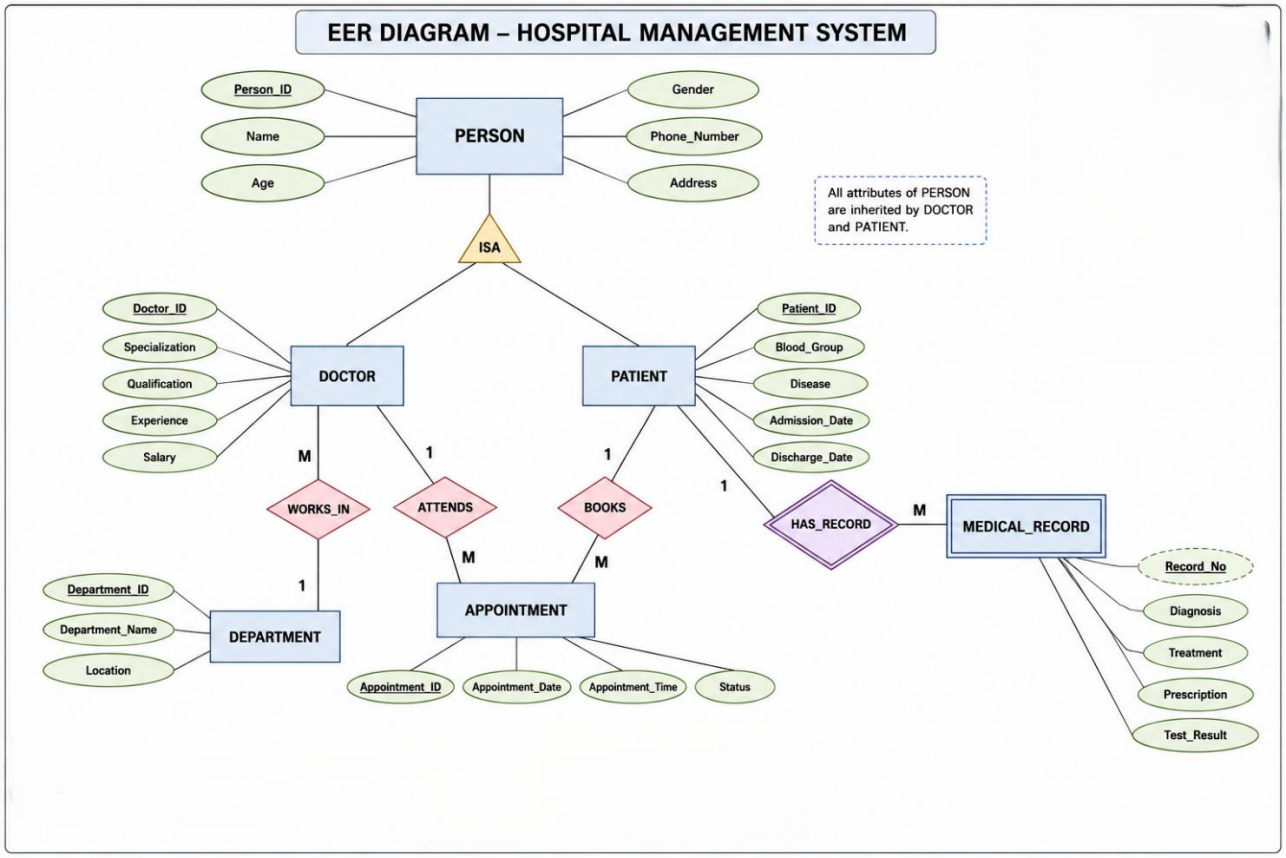
PATIENT → APPOINTMENT

- One patient can book many appointments.
- Each appointment belongs to one patient.
- Cardinality: **1 : M**

PATIENT → MEDICAL_RECORD

- One patient has many medical records.
- A medical record belongs to only one patient.
- Medical_Record is a **Weak Entity** identified by the Patient.
- Cardinality: **1 : M**

3. EER Diagram



4. Explanation of Inheritance

Inheritance allows subclasses to inherit common attributes from a superclass.

In this EER model:

- **Person** is the superclass.
- **Doctor** and **Patient** are subclasses.
- Both inherit:
    - Person_ID
    - Name
    - Age
    - Gender
    - Phone_Number & Address

The subclasses also contain their own unique attributes.

This reduces redundancy because common information is stored only once.

5. Explanation of Weak Entity

A **Weak Entity** does not have a complete primary key of its own and depends on another entity for identification.

In this model:

- **Medical_Record** is the weak entity.
- It depends on **Patient**.
- A medical record cannot exist without a patient.
- **Record_No** acts as a partial key and is combined with the Patient's key for unique identification.

6. Advantages of this EER Design

- Eliminates data redundancy through inheritance.
- Represents real-world hospital operations accurately.
- Improves data consistency.
- Supports efficient database management.
- Makes future expansion easier.
- Clearly models dependent entities using weak entities.

7. Applications

This EER model can be used in:

- Hospitals
- Multi-speciality Clinics
- Medical Colleges
- Diagnostic Centers
- Healthcare Information Systems

Conclusion

The Hospital Management System EER model uses **inheritance** by creating **Doctor** and **Patient** as subclasses of **Person**, allowing common information to be shared efficiently. It also includes **Medical_Record** as a **Weak Entity**, which depends on the **Patient** entity for identification. This design minimizes redundancy, improves data integrity, and accurately represents real-world hospital operations.

Q5. Explain the Role of a Database Administrator (DBA) and the Responsibilities Associated with the Position.

Introduction

A **Database Administrator (DBA)** is a professional responsible for managing, maintaining, securing, and optimizing databases in an organization. The DBA ensures that the database operates efficiently, remains secure, and is always available for users and applications. A DBA plays a vital role in maintaining data integrity, improving database performance, and protecting valuable organizational data.

Definition of DBA

A **Database Administrator (DBA)** is a person who is responsible for the installation, configuration, maintenance, security, backup, recovery, and performance of a Database Management System (DBMS). The DBA ensures that authorized users can access data safely while preventing unauthorized access.

Roles of a DBA

A Database Administrator performs several important roles in an organization:

1. Database Installation and Configuration

- Installs DBMS software such as MySQL, Oracle, SQL Server, or PostgreSQL.
- Configures the database according to organizational requirements.

2. Database Design and Maintenance

- Creates databases and database objects.
- Maintains tables, indexes, views, and relationships.
- Ensures proper database structure.

3. User Management

- Creates user accounts.
- Assigns user roles and privileges.
- Controls user authentication and authorization.

4. Database Security

- Protects the database from unauthorized access.
- Implements passwords, encryption, and access control.
- Monitors security threats and vulnerabilities.

5. Backup and Recovery

- Performs regular database backups.
- Restores data after accidental deletion, hardware failure, or disasters.
- Ensures business continuity.

6. Performance Monitoring and Optimization

- Monitors database performance.
- Optimizes SQL queries.
- Creates indexes to improve search speed.
- Removes unnecessary data and improves efficiency.

7. Data Integrity Management

- Ensures that stored data is accurate and consistent.
- Enforces primary keys, foreign keys, and constraints.
- Prevents duplicate or invalid data.

8. Storage Management

- Manages database storage space.
- Monitors disk usage.
- Allocates additional storage when required.

9. Troubleshooting and Problem Solving

- Identifies database errors.
- Resolves performance issues.
- Fixes database failures quickly to minimize downtime.

10. Database Documentation

- Maintains documentation of database structures, configurations, and procedures.
- Helps developers and future administrators understand the database.

Responsibilities of a DBA

The major responsibilities of a Database Administrator include:

- Installing and configuring the DBMS.
- Designing and maintaining databases.
- Creating and managing users.
- Implementing database security.
- Performing backup and recovery.
- Monitoring database performance.
- Optimizing queries and indexes.
- Ensuring data integrity.
- Managing storage efficiently.
- Troubleshooting database problems.
- Maintaining database documentation.
- Supporting developers and end users.

Importance of a DBA

A Database Administrator is important because the DBA:

- Ensures continuous availability of the database.
- Protects sensitive organizational data.
- Improves database performance.
- Prevents data loss through regular backups.
- Maintains data accuracy and consistency.
- Supports efficient business operations.

Real-Life Example

Consider a **Hospital Management System**.

The DBA performs the following tasks:

- Creates databases for Patients, Doctors, Appointments, and Medical Records.
- Grants access permissions to doctors, nurses, and receptionists.
- Performs daily backups of patient records.
- Restores data if the server crashes.
- Optimizes database performance so patient information can be retrieved quickly.
- Ensures confidential patient data is protected from unauthorized access.

Conclusion

A **Database Administrator (DBA)** is responsible for managing the complete database environment. The DBA installs, secures, monitors, backs up, recovers, and optimizes databases while ensuring data integrity, security, and availability. A skilled DBA helps organizations maintain reliable, efficient, and secure database systems that support daily operations and decision-making.